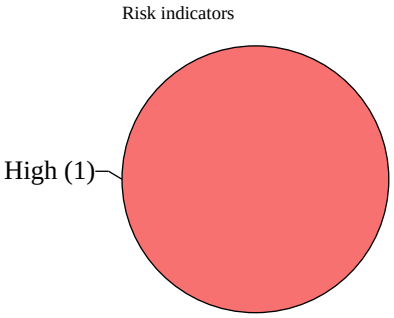
CodeGuard AI: Code Similarity Report

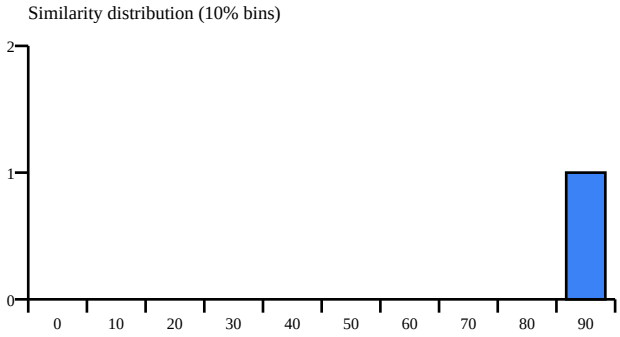
Generated 2026-10-23 10:47. 1 pair(s) compared.

Similarity scores are indicators for human review. They are not proof of plagiarism: shared assignment templates, common algorithms, boilerplate and short programs can all produce high scores, and copied code that was heavily restructured can score low.

Analysis summary

1 pair(s) from 2 file(s) were compared. 1 pair(s) are in the High indicator band, 0 Moderate and 0 Low; the average similarity is 100.0%. The highest pair is ansh__python_programming_assignment_1.py and riya__python_programming_assignment_1.py at 100.0%.





Ranking

File A	File B	Overall	Risk indicator	Matched lines	Blocks
ansh__python_programming_assignment_1.py	riya__python_programming_assignment_1.py	100.0%	High	20	1

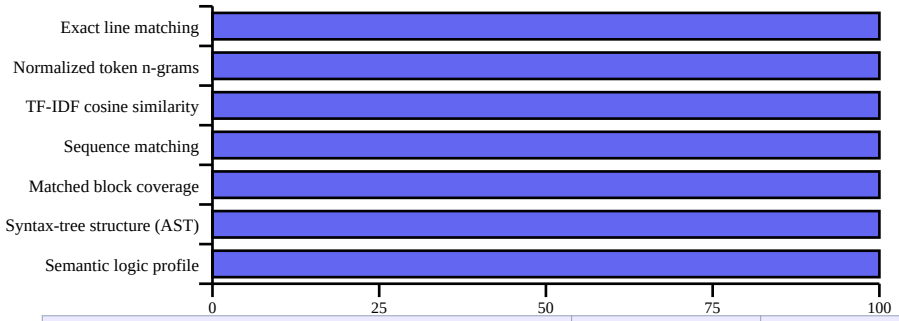
ansh__python_programming_assignment_1.py vs riya__python_programming_assignment_1.py

100.0% High similarity indicator. 20 matching line(s) in 1 block(s). Coverage: A 100.0%, B 100.0%.

Method: Python AST + normalized token analysis. Python syntax trees were compared in addition to the lexical, sequence and semantic signals.

Why this score

- Overall 100.0% (High indicator) is a weighted blend of 7 signals. Largest contributions: Matched block coverage 100%, Semantic logic profile 100%, Syntax-tree structure (AST) 100%.
- 1 matched block(s) cover 20 of 20 comparable lines in A (100%) and 20 of 20 in B (100%). Largest: A lines 4-30 matches B lines 4-30 (20 lines).
- Block types: 1 identical.
- 100% of comparable lines are identical after removing comments and whitespace.
- Syntax-tree structure overlaps 100%; 2 function(s) have matching structure (divide ~ divide, main ~ main).
- Excluded as non-evidence (false-positive reduction): A: 3 entry point; B: 3 entry point.



Signal	Score	Weight
Exact line matching	100.0%	10%
Normalized token n-grams	100.0%	12%
TF-IDF cosine similarity	100.0%	10%
Sequence matching	100.0%	12%
Matched block coverage	100.0%	20%
Syntax-tree structure (AST)	100.0%	18%
Semantic logic profile	100.0%	18%

Matched blocks

#	Lines in A	Lines in B	Comparable lines	Token identity	Kind
1	4-30	4-30	20	100.0%	identical

Token identity compares literal tokens. Blocks of kind 'semantic' match by canonical logic form (equivalent steps written differently, possibly in another language), so a low token identity is expected for them.

Matched code snippets (A on the left, B on the right)

Section 1: identical, 20 lines, token identity 100%

```
4   def add(a, b):
5       return a + b
6
7   def subtract(a, b):
8       return a - b
9
10  def multiply(a, b):
11      return a * b
12
13  def divide(a, b):
14      if b == 0:
15          return "Error: Cannot divide by zero."
    ... 15 more line(s)
```

```
4   def add(a, b):
5       return a + b
6
7   def subtract(a, b):
8       return a - b
9
10  def multiply(a, b):
11      return a * b
12
13  def divide(a, b):
14      if b == 0:
15          return "Error: Cannot divide by zero."
    ... 15 more line(s)
```

Functions with matching syntax-tree structure

Function in A	Lines	Function in B	Lines	Structure match
divide	13-16	divide	13-16	100.0%
main	18-30	main	18-30	100.0%

Unique code (no match in the other file)

A: 0 comparable line(s) in ranges none.

B: 0 comparable line(s) in ranges none.

Ignored as non-evidence: A: 3 entry point, 1 imports, braces and docstrings; B: 3 entry point, 1 imports, braces and docstrings.

Methodology and limitations

Exact line matching: Dice overlap of code lines that are identical after comments and whitespace are removed. Sensitive to copy-paste; blind to renaming.

Normalized token n-grams: Dice overlap of 4-token sequences in which identifiers, numbers and strings are replaced by placeholders. Resistant to renaming and reformatting; also matches generic language idioms.

TF-IDF cosine similarity: Cosine similarity of TF-IDF vectors built from identifiers, literals and structural token trigrams. Rare shared terms weigh more than common ones when several files are analysed together.

Sequence matching: difflib SequenceMatcher ratio over the ordered, identifier-normalized code lines. Rewards code that keeps its order.

Matched block coverage: Share of significant lines covered by matched blocks found with Greedy String Tiling, which also finds blocks that were moved.

Syntax-tree structure (AST): Dice overlap of node-type trigrams of the syntax tree (names, literal values and docstrings ignored). Python uses the real ast module; Java, C, C++ and JavaScript use a lightweight statement-level parser that builds an approximate tree. Same-language pairs only.

Semantic logic profile: Local semantic fingerprint: code is rewritten into a canonical form (compound assignments expanded, loop styles unified, types and modifiers dropped, I/O and collection calls mapped to common names) and compared statement by statement. Finds the same logic written with different variable names, loop styles or even languages. This is a rule-based model that runs fully offline, not a neural network.

The overall score is a weighted mean of the available signals; weights are configurable and re-normalized when a signal does not apply. Risk levels use configurable thresholds and are not universally valid cut-offs. Comments, whitespace, import lines, brace-only lines, standard boilerplate, instructor template code and lines common to most submissions are excluded. Java, C, C++ and JavaScript structure comes from a lightweight parser, and the semantic signal is a rule-based model, not a neural network. No automated method can detect every form of plagiarism.